

SPECIAL ISSUE PAPER

Assisting the continuous improvement of Scrum projects using metrics and Bayesian networks

Mirko Perkusich | Kyller Costa Gorgônio | Hyggo Almeida |
Angelo Perkusich

Embedded Systems and Pervasive Computing
Laboratory, Federal University of Campina
Grande, Campina Grande, 58109-900, PB,
Brazil

Correspondence

Mirko Perkusich, Embedded Systems and
Pervasive Computing Laboratory, Federal
University of Campina Grande, Campina
Grande, PB 58109-900, Brazil.
Email: mirko.perkusich@embedded.ufcg.edu.br

Abstract

Scrum is a simple process to understand, but hard to adopt. Therefore, there is a need for resources to assist on its adoption. In this paper, we present the process followed to build a *Bayesian network* to assist on the assessment of the quality of the software process in the context of *Scrum* projects. The model provides data to help *Scrum Masters* lead the improvement of business value delivery of *Scrum* teams. The process is divided into 2 phases. In the first phase, we built the *Bayesian network* based on expert knowledge extracted from the literature and experts. We used a top-down approach and reasoning to define the key metrics necessary to build the models and their relationships. In the second phase, we updated the *Bayesian network* based on limitations of the first version. We validated the *Bayesian network* inferences with 10 simulated scenarios. Comparing both versions, for all scenarios, we improved the accuracy of the inferences. Therefore, we concluded that the *Bayesian networks* adequately represent *Scrum* projects from the viewpoint of the *Scrum Master*. Finally, the model built is in conformance with agile methods tailoring and can be adapted to any *Scrum* team.

KEYWORDS

agile methods, Bayesian networks, continuous improvement, KaizenScrum

1 | INTRODUCTION

One of the reasons that agile methods have recently become popular on software development is because they focus on improving collaboration between developers and clients and rapidly responding to change of requirements. The use of agile methods is increasing to satisfy the need to respond to fast moving market demand and to gain market share.¹ According to a study² that collected data from over 4000 practitioners in 2014, the top 3 benefits of agile development are the ability to manage changing priorities and increase of team productivity and project visibility. According to the same study, *Scrum* dominates as the agile development process of choice with 56% of preference. Given this, in this work we focus on *Scrum* projects.

For its goals to be achieved, a key concept in agile software development is continuous improvement (i.e., Kaizen). In fact, 1 of the 12 principles of the Agile Manifesto* states that, at regular intervals, the team reflects on how to become more effective and then tunes and adjusts its behavior accordingly.

Scrum, which is based on an empirical process control theory,³ describes an event, namely, *Sprint Retrospective Meeting*, and a role, namely, *Scrum Master*, to support the project teams' continuous improvement. The *Sprint Retrospective Meeting* occurs at the end of every *sprint* (i.e., iteration) as an opportunity for the team to inspect itself and create a plan for improvements.³ The *Scrum Master* is responsible to ensure that the team adheres to *Scrum* theory, practices, and rules. During the *Scrum Retrospective Meeting*, the *Scrum Master* participates as a peer team member from the accountability over the *Scrum* process. Furthermore, the *Scrum Master* encourages the team to improve, within the *Scrum* framework, its development process, and practices to make it more effective and enjoyable. We present an overview on *Scrum* in Section 2.1.

On the other hand, even though *Scrum* is simple to understand, it is hard to master.^{3,4} It requires self-organizing, cross-functional teams and collaboration between business and development teams to develop products incrementally. Especially in companies with former waterfall approach, these factors might lead agile to fail. According to VersionOne,² 44% of respondents pointed to lack of experience with agile methods as the main cause for agile to fail and 42% to company

* <http://www.agilemanifesto.org>

philosophy or culture at odds with core agile values. As barriers to agile adoption, 44% pointed to the ability to change organizational culture, 35% to not enough personnel with the necessary agile experience, and 32% to preexisting waterfall framework.

Given this, there is a need for resources to assist on the adoption and continuous improvement of agile methods. Because agile is an empirical approach, it is necessary to define metrics to diagnose and improve the processes that produce business value. On the other hand, solely defining metrics is not enough. It is necessary to instrument the *Scrum Master* with data to, given the defined metrics, assess and predict the quality of artifacts (e.g., product backlog) and resources (e.g., development team).

Our solution is to combine the definition of metrics with *Bayesian networks*. We use this technique because it explicitly treats uncertainty, inherent to software process,⁵ by using random variables and probability functions. Considering probability allows handling missing data and modeling the confidence of assessments, which is a requirement in our context. Furthermore, they are flexible to handle different types of data and be easily modified, which is necessary to conform to process tailoring. For instance, it can model the causal relationship between objective (e.g., test code coverage) and subjective metrics (e.g., team motivation) of the software development process, which can be measured using different scales. Finally, it can be defined given the existing data or by eliciting data from experts, which is our case.

In software engineering context, this technique has been used for several purposes such as to predict the velocity of an agile project,⁶ predict the delay of software maintenance projects,⁷ and analyze software project risk.⁸ We present an overview of the usage of *Bayesian networks* in software engineering in Section 2.3.

In Perkusich et al.,⁹ we presented the first version of a *Bayesian network* that models the software process of *Scrum* projects by considering artifacts, managerial and technical practices, and team factors from the viewpoint of the *Scrum Master*. In Perkusich et al.,¹⁰ we presented a procedure to apply the model's usage into the team's process and presented results of industrial case studies in 2 projects. On the other hand, in neither of the papers, we presented details regarding the process used to identify the *Bayesian network*'s variables and define their relationship, which we present in this paper.

This paper provides 3 contributions. First, we present a model to represent the relationship between the *Scrum* process factors identified through literature review. Second, we present the process to build an executable *Bayesian networks*-based model to assist on assessing the software processes of *Scrum* projects. Last, we present the process applied to validate the inferences of the *Bayesian network*.

Comparing with our prior works,^{9,10} we present 3 novelties. First, the process of identifying the variables of the model has never been presented before. Second, we discuss limitations of the model presented in Perkusich et al.¹⁰ and the process executed to develop a new version of the *Bayesian network*, which is composed of new practices and objective metrics. Last, we present results regarding the validation process of the new *Bayesian network*, in which we used simulated scenarios.

This paper is organized as follows: in Section 2, we present relevant literature review focusing on *Scrum* and *Bayesian networks*; in Section 3 we present the methodology for building the proposed model; in Section 4, we present details regarding the process of building the directed acyclic graph (DAG) of the first version of the *Bayesian network*; in Section 5, we present details regarding the process to build and validate the the second version of the *Bayesian network*; in Section 6, we present our conclusions, current limitations, and future works.

2 | BACKGROUND

2.1 | Scrum

Scrum is the most used agile framework in the industry² and it is common to combine it with other agile frameworks such as *eXtreme Programming*¹¹ and *Kanban*¹² (a process called *ScrumBan*). *Scrum* is an iterative and incremental process to optimize the ability to foresee and control risks. The *Scrum* process is sustained by 3 pillars: transparency, inspection, and adaptation.¹³ At the end of each iteration, called *sprint*, a functional product increment is delivered to be verified and validated by the stakeholders. Important aspects of the process such as acceptance criteria must be visible (i.e., transparent) to all stakeholders. The stakeholders should inspect the artifacts and progress of the project frequently to detect any undesired variability. Finally, the project should be adapted to support changes. We present an overview of the *Scrum* process in Figure 1.

There are 2 main artifacts: product backlog and sprint backlog. The product backlog is an ordered list that represents the product's features, requirements, improvements, and bug fixes. It should be ordered given value, risk, business priority, and necessity. Furthermore, it is emergent and should adapt to business changes.¹⁴ The sprint backlog is composed of product backlog items that were allocated to a given sprint.¹³

There are 3 roles: *Product Owner*, *Scrum Master*, and developer.¹³ The *Product Owner* is responsible for maximizing the product's value. He

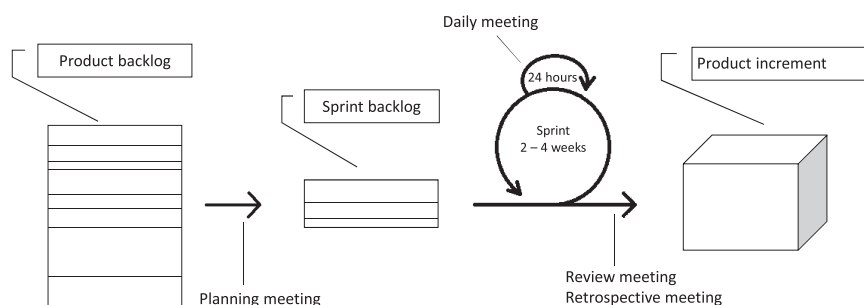


FIGURE 1 Overview of Scrum

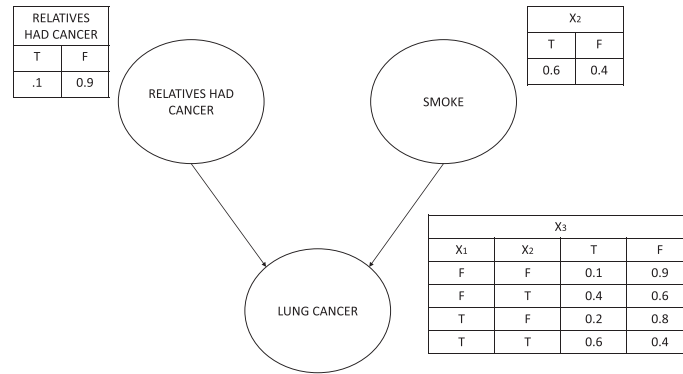


FIGURE 2 A Bayesian network Bayesian network example

or she should serve as an interface between the technical team and the business team and is responsible for managing the product backlog. The *Scrum Master* serves as a servant-leader and is responsible to ensure that *Scrum's* theories, rules, and practices are correctly applied in the project. The developers are responsible for executing any activity related to delivering the product increment at the end of the sprints (e.g., design, implement, and test).

There are 4 essential meetings: planning, daily, review, and retrospective.¹³ The planning meetings occur at the beginning of each sprint to define its work and goal. The daily meetings occur daily to inspect the progress of the sprint and synchronize the team's work to mitigate risks. The review meetings occur at the end of each sprint to inspect the produced product increment and, if necessary, to adapt the product backlog to changes requested by the business team. The retrospective meetings occur after the review meetings to assess the interactions between people, relationships, processes and tools. During this meeting, the team identifies problems, defines action points (i.e., corrective and preventive actions), and defines a plan to apply the action points.

2.2 | Bayesian networks

Bayesian networks are probabilistic graph models and are used to represent knowledge about an uncertain domain.¹⁵ A *Bayesian network*, B , is a DAG that represents a joint probability distribution over a set of random variables V .¹⁶ The network is defined by the pair $B = \{G, \theta\}$. G is the DAG in which the nodes $X_1, \dots, X_n$ represent random variables and the arcs represent the direct dependencies between these variables. θ represents the set of the probability functions. This set contains the parameter $\theta_{x_i|\pi_i} = P_B(x_i|\pi_i)$ for each x_i in X_i conditioned by π_i , the set of the parameters of X_i in G . Equation (1) presents the joint distribution defined by B over V .

$$P_B(X_1, \dots, X_n) = \prod_{i=1}^n P_B(x_i|\pi_i) = \prod_{i=1}^n \theta_{x_i|\pi_i}. \quad (1)$$

We present an example of a *Bayesian network* in Figure 2. The ellipses represent the nodes and the arrows represent the arcs. The probability functions are usually represented by tables. Even though the arcs represent the causal connection's direction between the variables, information can propagate in any direction.¹⁷

Bayesian networks have many advantages such as suitability for small and incomplete data sets, structural learning possibility, combination of different sources of knowledge, explicit treatment of uncertainty, support for decision analysis, and fast responses.¹⁸ Therefore, they are applied to support systems with uncertainty.¹⁹ *Bayesian networks* have been used for several expert systems such as assisting in safety decision making in complex project environments²⁰ and to predict performance in innovation projects given their transformational leadership characteristics.²¹

There are 2 challenges to build *Bayesian networks*: building the DAG and defining the probability functions.²² To assist on building the DAG, Fenton et al²³ present a set of idioms, which are *Bayesian network* fragments that represent the graphical part of generic types of uncertain reasoning. In this paper, we will only present the idiom that we use in our method: synthesis. Synthesis idiom models the synthesis or combination of many nodes into 1 node for the purpose of organizing the *Bayesian network*. We used synthesis idiom to decompose the initial factors into observable ones. We present in Figure 3 an example of this idiom extracted from Fenton and Neil.²⁴

A common application of the synthesis idiom is to reduce the effects of combinatorial explosion, which is the case when a node has too many parents, by divorcing the parents. According to Fenton and Neil,²⁴ ideally a node should have at most 3 parents. For instance, say that assessing *Tests results* depends on *Occurrences of failures*, *Severity of faults*, *Tester experience*, and *Testing effort*. Considering that each node has 5 states, the Node Probability Table (NPT) of *Test results* would have 5^4 cells. To reduce this complexity, we could add 2 synthetic nodes: *Reliability* and *Test quality*. *Reliability* is defined in terms of *Occurrences of failures* and

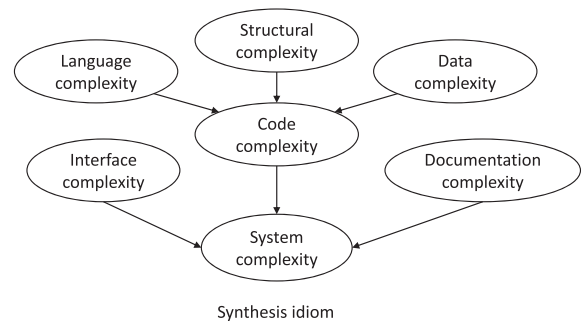


FIGURE 3 Example of synthesis idiom

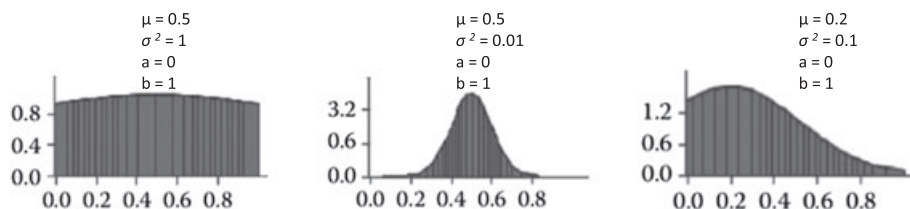


FIGURE 4 TNormal examples

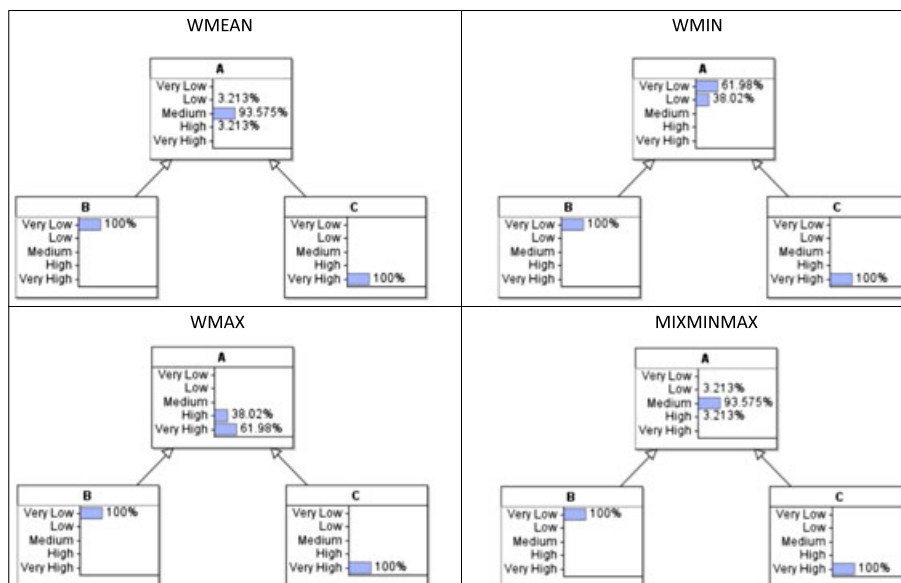


FIGURE 5 TNormal examples

Severity of faults. *Test quality* is defined in terms of *Tester experience* and *Testing effort*. Finally, *Tests results* is defined only in terms of *Reliability* and *Test quality*. With this new configuration, the total number of cells to define the NPTs of *Test results*, *Reliability*, and *Test quality* is 3×5^2 cells, reducing the complexity in 88%.

The *Bayesian network's* probability functions are usually represented as NPTs. In many cases, there are not enough data to define them and it is necessary to collect them from domain experts. However, manually defining the NPTs through domain experts can become unfeasible depending on the number of nodes and states, because the complexity grows exponentially.

To reduce the effort of defining NPTs through domain experts, Fenton et al.²³ proposed the concept of ranked nodes. It is based on the doubly truncated normal distribution (TNormal) limited in the $[0,1]$ region. This distribution is based on 4 parameters: μ , mean (i.e., central tendency); σ^2 , variance (i.e., confidence in the results); a , lower bound (i.e., 0); and b , upper bound (i.e., 1). This distribution enables us to model a variety of shapes (i.e., relationships) such as a uniform distribution, achieved when $\sigma^2 = \infty$, and highly skewed distributions, achieved when $\sigma^2 = 0$. We show TNormal examples in Figure 4.

In the approach presented in Fenton et al.,²³ μ is defined by a weighted function of the parent nodes. There are 4 weighted functions: weighted mean (WMEAN), weighted minimum (WMIN), weighted maximum (WMAX), and mixture of WMIN and WMAX (MIXMINMAX). According to the authors, these functions are enough to represent the types of relationship necessary for defining NPT. We show example

NPT calculated with these functions in Figure 5. In the examples presented in Figure 5, WMEAN and MIXMINMAX have the same values. The difference between the 2 functions is that WMEAN calculates the weighted mean of the parent node's values (i.e., the weights are set for the parent nodes) and MIXMINMAX mixes the minimum and maximum functions (i.e., the weights are set for the functions).

To define which function should be used, the model developer must perform "what if" analysis with the expert by defining questions and collecting answers to define a truth table. The model developer must analyze the answers and define the most appropriate function. The variance is defined empirically and it should reflect the expert's confidence in the results.²³ We show an example of questions and answers in Table 1, in which the node C has 2 parents, A and B. In the example shown, the most appropriate function is WMIN.

We used ranked nodes, because we did not have enough data to build the Bayesian network using batch learning. Therefore, we needed to elicit data from an expert. By using ranked nodes, the effort to elicit data

TABLE 1 Example of truth table to define the weighted function for μ

A	B	C
Very high	Very high	Very high
Very low	Very low	Very low
Very low	Very high	Very low
Very high	Very low	Low

is reduced. Furthermore, the solution is implemented in AgenaRisk,[†] a tool to execute Bayesian networks.

2.3 | Bayesian networks applied to software engineering

According to the Maxim of Uncertainty in Software Engineering,⁵ uncertainty is inherent and inevitable in software development processes and products. As a corollary to the Maxim of Uncertainty in Software Engineering, Ziv and Richardson⁵ propose that software uncertainties should be modeled and managed explicitly through uncertainty modeling techniques such as *Bayesian networks*. Ziv and Richardson⁵ state that *Bayesian networks* offer a mathematically sound computational model for uncertainty reasoning and that their graph structure matches software systems. Furthermore, because software practitioners have experience with tree structures, the graphical structure of *Bayesian networks* facilitates communication with them and facilitate analysis and modifications. Therefore, *Bayesian network* has been applied to many areas in software engineering such as risk management and product quality prediction.

Fan and Yu,²⁵ Hearty et al,⁶ Fenton et al,²⁶ Jeet et al,²⁷ and Hu et al⁸ modeled software processes to support risk management. Fan and Yu²⁵ built a model capable of predicting potential risks, identifying the source of risks, and supporting dynamic resource adjustment. Fenton et al²⁶ showed how to use a *Bayesian network* to predict software defects and perform “what if” scenarios. Jeet et al²⁷ built a model to estimate the impact of low productivity on the schedule of a software development. They used interviews and historical data to build the model. Hu et al⁸ proposed a risk identification framework for outsourced software projects. These works present models that have different purposes than the procedure we propose.

Abouelela and Benedicenti²⁸ and Jeet et al²⁹ modeled software processes to support quality management. Abouelela and Benedicenti²⁸ built a model to predict the rate of defects of an XP project. Furthermore, the model predicts the duration of the project. Thus, the proposed model allows its users to determine if an XP project will succeed. The inputs of the model are as follows: information on requirements, size of the team, *pair programming* usage, *Test-Driven Development* usage, client availability, team velocity, and the number of defects. Jeet et al²⁹ built a model to detect the number of defects in a software development project. The rate of defects and the project manager's judgments are used for predictions and support in managing the number of defects. These works have limited scope and focus on making predictions to support quality management.

Pendharkar et al³⁰ and Melo and Sanchez⁷ modeled software processes to support project planning. Pendharkar et al³⁰ used *Bayesian networks* to predict the effort necessary to complete a software development project using historical data and subjective estimations of experts. Melo and Sanchez⁷ presented a *Bayesian network* for maintenance project delays based on the experience of specialists and a corresponding tool to help in managing software maintenance projects.

Settas et al,³¹ Stamelos,³² and Stamelos et al³³ modeled software processes to support other project management activities.

Settas et al³¹ and Stamelos³² used *Bayesian networks* to help managerial decision making by modeling software project management antipatterns. Stamelos et al³³ modeled the uncertainties of factors to estimate software productivity.

Hearty et al⁶ used a *Learning Dynamic Bayesian network* model to predict project velocity in XP. It can learn from project data to make quantitative effort predictions and risk assessment. According to the authors, their approach is also suitable for *Scrum*. One of the factors used by the model is *process effectiveness*. Because it only considers a few XP practices, it is one of the model's limitations. As shown in Sections 4 and 5, our models consider *Scrum* roles, events, and agile software development and project management best practices to assess the process quality. Given this, as presented in Section 6, in future works, we intend the models created using our method with the model presented in Hearty et al⁶ to combine process improvement and project management information.

Nagy et al³⁴ used *Bayesian networks* to build a model to monitor the health of an agile project considering its key factors. The proposed model was built using the knowledge of experts, but Nagy et al³⁴ did not present the elicitation process, experts background, and justifications regarding the identified key agile factors. Furthermore, the model was not validated. Given its weaknesses and the lack of instructions on how to use it, we believe that it has limited contribution.

In this paper, we present the steps executed to, as shown in Sections 4 and 5, construct a *Bayesian network* to assist on the continuous improvement of *Scrum*-based projects considering *Scrum* roles, events, and agile software development and project management best practices, which is a novel approach.

3 | METHODOLOGY

Our goal was to build a generic process assessment model to assist on the continuous improvement of *Scrum* software development projects by considering *Scrum*'s roles, events and agile software development and project management best practices. By generic, we mean that it considers the official *Scrum* roles, events, rules and management practices as described in the *Scrum Guide*³ and recommended technical practices. We consider the *Scrum Guide* as the official document that describes *Scrum*, because it is authored by the creators of *Scrum*: Jeff Sutherland and Ken Schwaber. The proposed model's goal is not to fit all organizations that use *Scrum*, because it is not realistic. As it occurs with the adoption of any software development method (e.g., Rational Unified Process or Extreme Programming), an organization might need to tailor it to meet its context and needs. Therefore, the model's goal is to consider “best practices” and minimize the effort necessary to adopt it by organizations.

We decided to follow the *Scrum Guide*, because it has been shown that deviations from it that are not well motivated and reasonable can lead to negative consequences such as reduced product quality. In Elooranta, Koskimies, and Mikkonen,³⁵ the authors show data collected from 18 teams in 11 companies resulting in 14 antipatterns on the adoption of *Scrum* such as having a *Product Owner* without authority and an unordered *Product Backlog*.

[†]www.agenarisk.com

On the other hand, monitoring only the factors identified through the Scrum Guide is not enough to reach our goal: guide the continuous improvement of the software process of teams that use *Scrum*. As discussed in Williams,³⁶ using only *Scrum* managerial practices on software development projects result in reduction of productivity caused by poor product quality. Therefore, we added agile technical practices to the generic model. As already discussed, some technical practices contained in the generic model might not make sense to a given context, but this is not a limitation of the model itself. It is a natural consequence of adopting any prescribed software development method.³⁷

Because our model is a *Bayesian network*, the problem of building it can be divided into (i) defining the DAG and (ii) the NPT. The DAG represents key software process factors (e.g., agile project management best practices) and their relationships in a *Scrum* software development project. A key software process factor is a factor related to the software process that the model developer judged as relevant for the model (e.g., *software validation*). The NPT quantifies through probabilities the relationships of the nodes.

On the DAG, each node represents a key factor, and there is an edge between 2 nodes whenever they relate to each other. Also, for each edge, the influenced key factor on the relationship is the edge's end point. Furthermore, each key factor has possible states and each one of them has an associated probability. Given that, as shown in Perkusich et al.,¹⁰ each key factor represents a set of tuples $N = \{(s_1, p_1), \dots, (s_{|N|}, p_{|N|})\}$, where s_i is a possible state and p_i is the associated probability. Also, we have the set of key factors $F = \{N_1, \dots, N_{|F|}\}$. Furthermore, we have the set of edges $R = \{(N_j, N_k) | N_j \subset F \wedge N_k \subset F\}$, where N_j is the initial point and N_k is the end point of the edge. Therefore, the problem is to find all elements of the sets F and R . To find all elements of the set F , we need to identify all key factors N_a , and for each key factor N_a , find all the possible states s_i and associated probability p_i , where $a \leq |F|$ and $i \leq |N_a|$. Finally, to find all elements of the set R , we need to identify all the relationships between the key factors, which are represented by the 2-tuples (N_j, N_k) , where N_j and $N_k \in F$.

To build the *Bayesian network*, we followed an engineering process based on Neil, Fenton, and Nielson.²² We show the main activities of the process in Figure 6. The process is divided into 2 phases. In the first phase, the steps are as follows: (i) specify requirements, (ii) define DAG, (iii) build the NPTs, (iv) validate Bayesian network inferences, and (v) evaluate Bayesian network's usage. In step (i), the requirements of the model were defined. As already discussed, the model must be generic and consider *Scrum*'s key process factors as described in the Scrum Guide and agile technical practices recommended by the literature. Furthermore, it must be built from the viewpoint of the *Scrum Master*; the input variables must be subjective metrics of observable factors and represent 1 *Scrum* team. In this paper, from the first phase, the scope is to present details regarding step (ii), which are presented in Section 4. This version of the Bayesian network was validated with simulated scenarios and evaluated with a case study in 2 projects. For details regarding the remaining steps, refer to Perkusich et al.^{9,38,10}

The second phase is composed of 3 steps: (vi) update the DAG, (vii) update the NPTs, and (viii) validate the Bayesian network inferences. In the second phase, we updated the Bayesian network given the limitations of the first version. The limitations are discussed in Section 4.3. We present the process to update and validate the Bayesian network in Section 5.

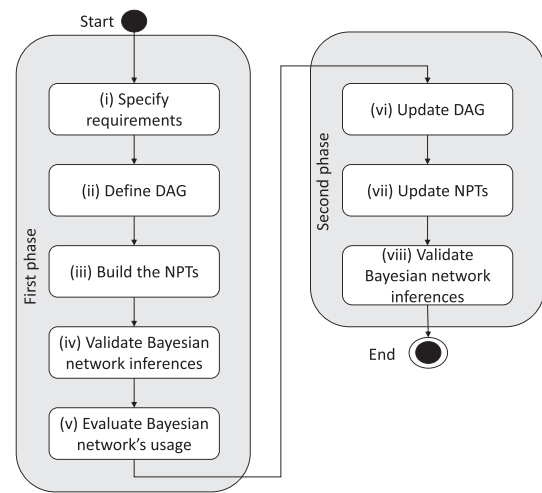


FIGURE 6 Process followed to build the *Bayesian network*. DAG indicates directed acyclic graph; NPT, Node Probability Table

4 | PROCESS TO BUILD THE FIRST VERSION OF THE DAG

The problem of defining the DAG was subdivided into 2: define the nodes and relationships and define the nodes' states. We solved the first problem by following a top-down approach in which we decomposed the top-level node, *Process quality*, into factors that we judged to be observable by the team's *Scrum Master*. To decompose the nodes, we used reasoning and data from the Scrum Guide, respected agile practitioners such as Cohn¹¹ and Pichler¹⁴ and scientific literature, as shown in Section 4.1. Afterwards, we presented the initial elements (i.e., key software process factors) to 11 experts and, using the Delphi technique,³⁹ refined the sets of elements during one-to-one discussions as shown in Section 4.2. Finally, we defined the nodes' states as shown in Section 4.2.

Given the popularity of *Object Oriented* design, to encapsulate the DAG complexity, we explain its construction using an analogy with *Object Oriented* paradigm in which dependent nodes in the DAG are treated as classes; independent nodes (i.e., observable factors) as primitives; and relationship between dependent nodes as class composition. We show an example of the analogy used in Figure 7. In the class diagrams, we use bold font to represent classes (i.e., objects).

4.1 | Initial definition of the DAG

Because our goal was to assess the process quality, we defined the first element of F : *Process quality*. As presented in Perkusich et al.,¹⁰ all software processes include 4 main activities: software specification, software design and implementation, software validation, and software evolution. In *Scrum*, the *Product Owner* is responsible for maximizing the value of the product.³ In other words, the *Product Owner* is responsible for the software specification. Furthermore, at the end of every *sprint* a product increment must be delivered. By assessing this artifact, we assess software design, implementation, and evolution. Finally, software validation is performed by increment inspection at the end of

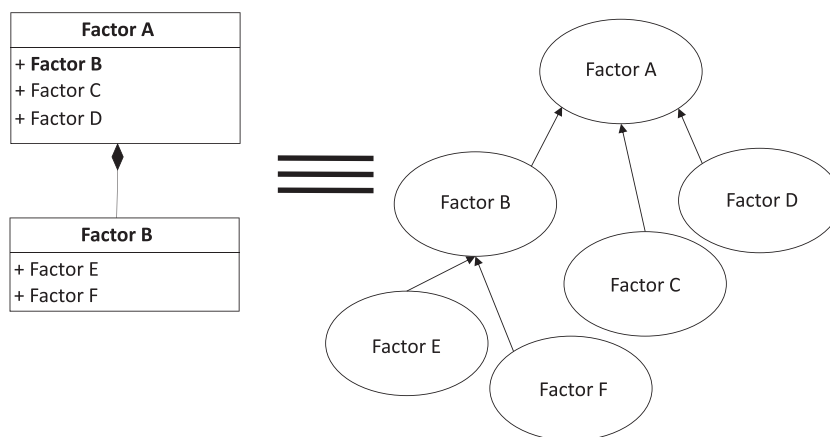


FIGURE 7 Analogy between *Object Oriented* paradigm and directed acyclic graph's tree structure

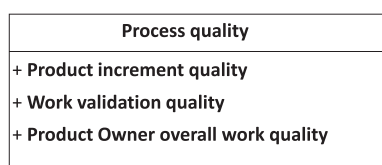


FIGURE 8 *Process quality* class diagram

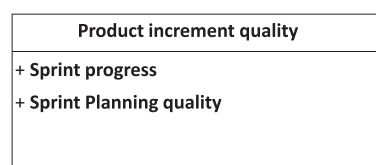


FIGURE 10 *Product increment quality* class diagram

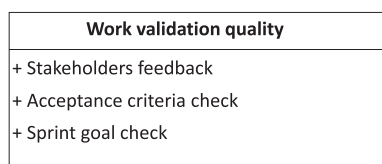


FIGURE 9 *Work validation quality* class diagram

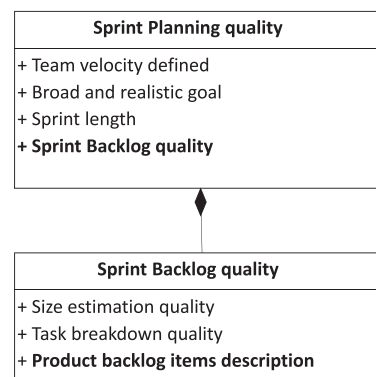


FIGURE 11 *Sprint planning quality* and *Sprint backlog quality* class diagrams

every sprint.³ Given that *Work validation quality*, *Product increment quality*, and *Product Owner overall work quality* are not directly observable, we considered them as classes. We show the resulting class diagram in Figure 8.

Afterwards, we defined the class *Work validation quality*. In *Scrum*, stakeholders validate the work (i.e., product) during *Sprint Review meetings*. During this meeting, the *Product Owner* verifies which items are done by checking their acceptance criteria, and checks if the *sprint* goal was reached and if the stakeholders collaborates with the *Scrum* team to analyze if the product attends the market needs.³ Not having *Sprint Review meetings* or not having the customers to attend it might lead to a lot of rework due to misunderstanding and development of wrong features.³⁵ The *Scrum Master* can observe if the activities described were executed successfully during *Sprint Review meetings*. Given this, we considered them as primitives. We show the *Work validation quality* class diagram in Figure 9.

For the stakeholders and *Product Owner* to be able to guide product development, the team delivers a potential shippable product (i.e., increment) at the end of every *sprint*.³ The delivered increment is a result of the activities performed during the *sprint* and its planning. We show the *Product increment quality* class diagram in Figure 10.

The *sprint* planning occurs during the *Sprint Planning meeting*. During this meeting, the *Sprint Backlog Items*, which are extracted from

the *Product Backlog*, are negotiated between the *Product Owner* and the developers³ to define the *Sprint Backlog quality*. The *Sprint Backlog's* quality depends on the *Product Backlog Items's* quality and the team's ability to decompose them into technical tasks⁴⁰ and estimate their size. Furthermore, the *sprint* length must be defined according to the project's environment (e.g., frequency of changes and stakeholders availability), have at most 4 weeks, and be fixed.^{3,35} *Velocity* is a metric used to assist the team during the *Sprint Planning meeting*, because it indicates its workload capacity per sprint.^{6,11,41} Finally, the *sprint goal*, which must be broad and realistic, is defined during the *Sprint Planning meeting* to give focus to the developers. We show the resulting class diagram in Figure 11, in which the class *Product Backlog item quality* is described later.

During the *sprints*, there are 3 factors that influence their progress: *Daily Scrum meetings*, *Sprint Backlog* stability and the development

team's competence. *Daily Scrum meetings* are essential to satisfy the *Scrum* principle that states that team must inspect and adapt constantly.³ Furthermore, *Scrum* assumes that the team is not disturbed during the *sprints*, because it interrupts the work flow of the team, consequently reducing its efficiency.³⁵ Therefore, the *Sprint Backlog*, which represents the *sprint*'s scope, must be stable and represent all the work performed by the team during the *sprint*. We show the *Sprint progress* class diagram in Figure 12.

Daily Scrum meetings improve communications, eliminate other meetings, identify impediments, highlight and promote quick decision-making, and improve the development teams' level of knowledge.³ They are time-boxed to 15 minutes and their purpose is to synchronize activities and create a plan for the next day, not to report status. During these meetings, team members must answer 3 questions: (i) What did I do yesterday that helped the team meet the *sprint goal*? (ii) What will I do today to help the team meet the *sprint goal*? and (iii) Do I see any impediment that prevents me or the team from meeting the *sprint goal*? According to Stray, Sjøberg, and Dyb⁴² this meeting's topics should be restricted to these 3 questions, with more focus on the last two. Self-justification during this meeting is an indicator of escalation of commitment⁴³ and technical discussions should be taken in a post meeting.⁴² We show the *Sprint progress* class diagram in Figure 13.

In *Scrum*, the development team is self-managed and cross-functional and must build the product incrementally.³ Self-management is considered a key factor to indicate an agile team's maturity.⁴⁴ Not having a cross-functional team might result in less committed team, staged development, slower feedback loops, and complex communication.³⁵ Given this, its competence depends on its teamwork quality, software engineering practices, and ability to continuously improve.¹¹ We defined that the teamwork quality depends on the following: members' personality, members' technical knowledge, motivation, collective ownership, team physical distribution, and team size. We decomposed the software engineering practices into code quality, inspection, and integration frequency. We show the resulting class diagram in Figure 14.

According to Cohn,¹¹ *Scrum* teams should use XP's software engineering techniques.⁴⁵ For the code to be inspected, *pair programming* is a recommended practice for agile teams to improve quality, productivity, and knowledge sharing.⁴⁶ On the other hand, with this purpose

Sprint progress
+ Development team competence
+ Daily Scrum quality
+ Sprint items and priorities stability

FIGURE 12 *Sprint progress* *Sprint progress* class diagram

Daily Scrum quality
+ Monitor sprint progress
+ All participants present
+ Answer the 3 basic questions
+ 15 minutes limit

FIGURE 13 *Daily Scrum quality* *Daily Scrum quality* class diagram

many agile teams use *peer code review*.⁴⁷ To guarantee code quality, teams should use *Test Driven Development*. If the team does not use *Test Driven Development*, it must execute another process that includes test automation, refactoring, and evolution of tests, which are popular agile practices³⁶ to maintain high product quality. To integrate code, many agile teams use *Continuous Integration*, which, according to Williams,³⁶ is the most popular agile practice. Given this, we added 5 elements to *F*: *Pair programming* and *Peer code review*, children of *Code inspection quality*; and, *Code refactoring*, *Automated tests* and *Tests coverage analysis*, children of *Code quality*. We show the resulting class diagram in Figure 15.

The *Product Owner* is the only responsible to manage scope and all stakeholders must respect his/her decisions.³ The *Product Owner* must have specific personal characteristics,⁴⁴ guarantee the *Product Backlog* quality, and track the project's progress.³⁵ According to Pichler,¹⁴ the *Product Owner* must be visionary and doer, leader and team player, empowered and committed, available and qualified, and communicator and negotiator. A *Product Owner* with incorrect personality and empowerment might cause unclear requirements and poor communication quality. Lack of progress visibility results in lack of team awareness.³⁵ We show the resulting class diagram in Figure 16.

The *Product Backlog* quality depends on its management and the *Release Plan* quality. The *Product Backlog* must be emergent, estimated, ordered, and properly detailed.³ By being emergent, it stays flexible to changes in requirements. By being estimated, it helps on the project tracking. By being ordered, it gives the team focus on time to market and deliver value to the client.³⁵ By being properly detailed, it facilitates communication between the team and customers.

In agile teams, a practice to foster motivation and collaboration is to have the release plan and product vision communicated to the whole team.³⁶ The *Release Plan* depends on the product vision quality, the description of the top 3 to 5 features, and the definition of the launch data.¹⁴ The product vision must have a broad and engaging goal, be clear and stable, short and concise, and describe the critical attributes to satisfy the customer needs.¹⁴ We show the resulting class diagram in Figure 17.

To order the *Product Backlog*, the *Product Owner* must consider business value, risks, and technical dependencies.³ Furthermore, the *Product Backlog Items* must be negotiable and testable; top items must be small enough to be implemented during 1 *sprint*, have clear and complete acceptance criteria, independent, and consider nonfunctional requirements.¹¹ We show the resulting class diagram in Figure 18.

This way, we finalized the initial analysis and built the model's first version. During this process, we identified sixty 7 variables: 22, dependent; and 46, independent.

4.2 | Discussions with specialists

We consulted a group of 11 experts to evaluate the results presented in Section 4.1. The group was composed of practitioners with experience as *Scrum Master*, *Product Owner*, agile project manager, agile coach, consultants, and team leaders in companies located in Brazil, USA, Europe, and India. To reach a consensus between the practitioners, because they were spread in different locations and it was not possible to meet with all simultaneously, we applied the Delphi technique.³⁹

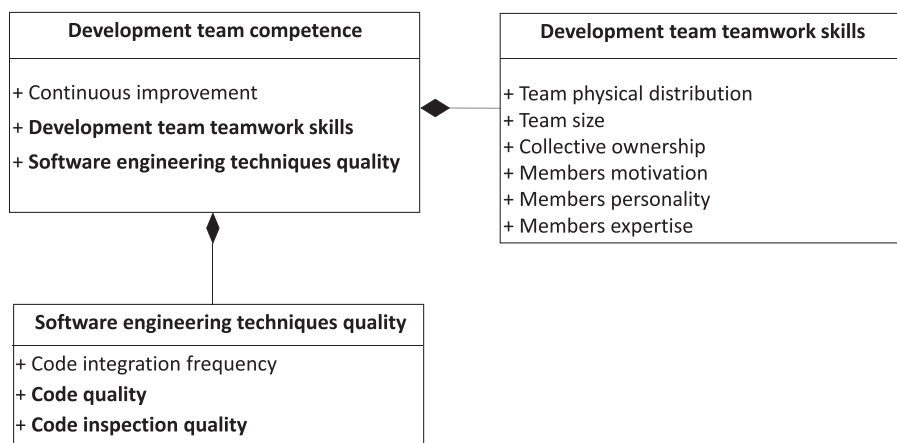


FIGURE 14 Development team competence, Development team teamwork skills, and Software engineering techniques quality class diagrams

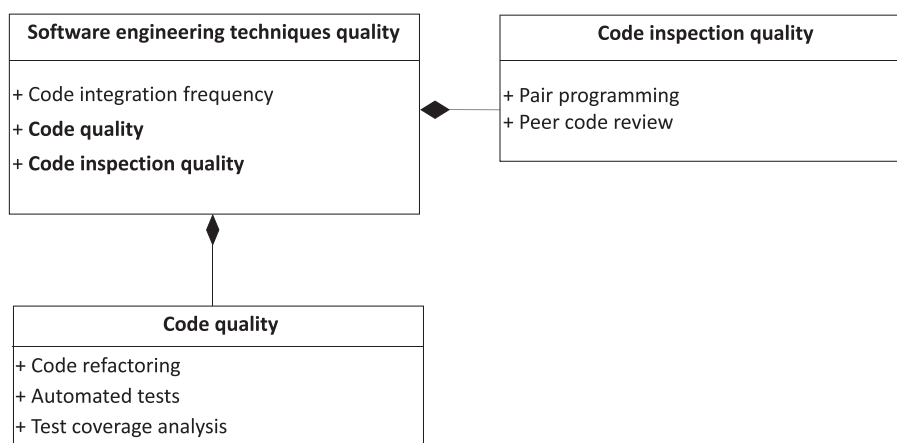


FIGURE 15 Software engineering techniques quality, Code quality, and Code inspection quality class diagrams



FIGURE 16 Product Owner overall quality and Product Owner personal characteristics class diagrams

The Delphi technique is a structured group communication process to conduct detailed discussion regarding a specific issue. We used an iterative process to apply the Delphi technique and aimed for reaching consensus for, at least, 7 of the experts. In the first iteration, we had one-to-one discussions with the experts. This discussion included

explaining the model's goal and the process that we used to identify the key process factors in *Scrum* projects and receiving their feedback. Afterwards, we consolidated the feedback of all the experts and developed, for the fragments of the DAG that had not reached a consensus, a set of possible options. In the second iteration, we sent a questionnaire containing 1 question for each fragment that had not reached a consensus asking the expert their opinion regarding the best option. At the end of the second iteration, we reached a consensus.

The DAG was modified after the practitioners' evaluation. According to 8 of them, in real projects, it is common for clients to be unavailable during *Sprint Retrospective meetings*. Therefore, their feedback is collected in separate meetings. We show the resulting *Work validation quality* class diagram in Figure 19.

According to 7 of the practitioners, code documentation should be considered a factor in code quality. In agile projects, scope changes frequently and having the code well documented makes changes easier to be executed. Furthermore, 9 practitioners stated that, for code inspection, static analysis tools are often used to automate this process. We present these modifications in Figure 20. After the practitioners' evaluation, *F* was composed of 73 variables: 21, dependent; and 50, independent. We show the final class diagram in Figure 21.

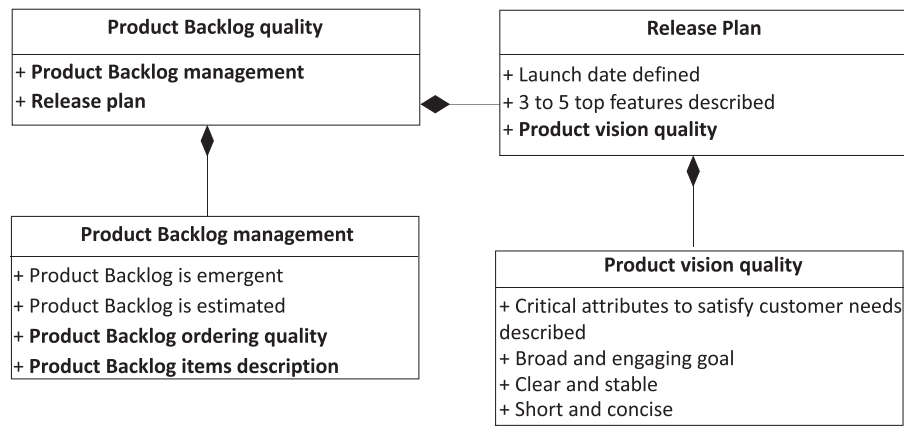


FIGURE 17 *Product Backlog quality*Product Backlog quality, Product Backlog management, Product Backlog items description and Release plan,Release plan, and Product vision qualityProduct vision quality class diagrams

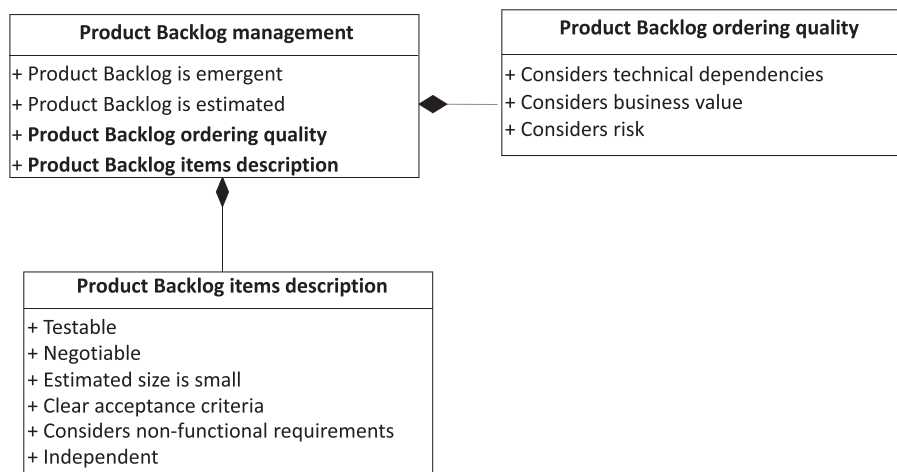


FIGURE 18 *Product Backlog ordering quality*Product Backlog ordering quality class diagram

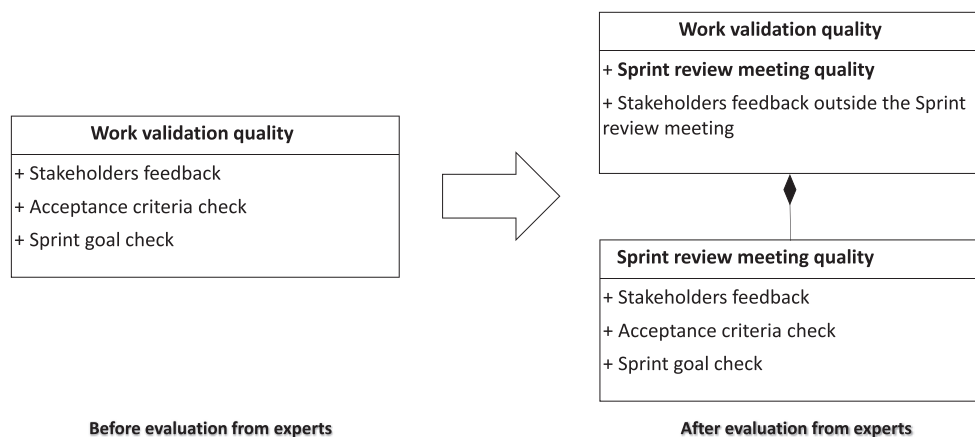


FIGURE 19 *Work validation quality*Work validation quality class diagram's modifications after practitioners' evaluation

Furthermore, we consulted the group of experts who participated in the Delphi regarding the number of states each node should have. After the discussions, we decided to use 3 states because it is quick and simple to use. Thus, we agreed that, given our context, it would not negatively influence the capabilities of the model. Because for all nodes we use an ordinal scale, the 3 states defined were as follows: *Bad*, *Moderate*,

and *Good*. We defined *High* as the state that represents the best case scenario for the given factor (i.e., node).

4.3 | Limitations

Even though the model's usage was successfully evaluated with an industrial case study with 2 projects,¹⁰ there are limitations and

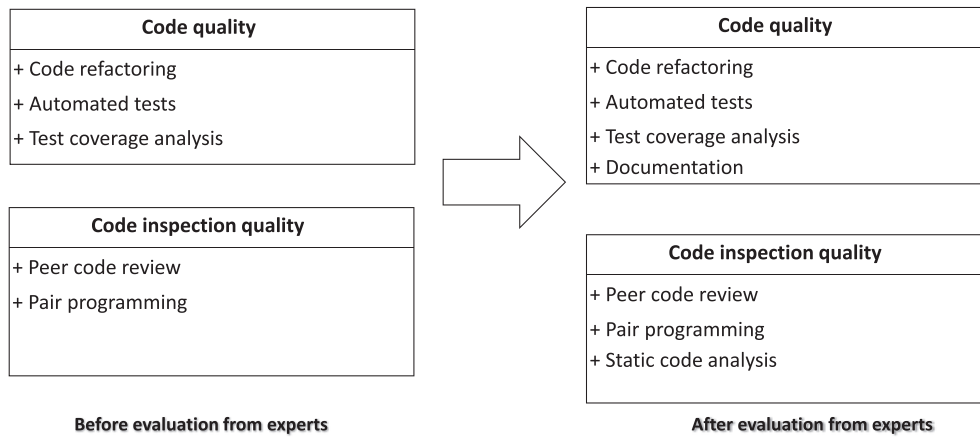


FIGURE 20 Code qualityCode quality and Sprint planning qualitySprint planning quality class diagrams' modifications after practitioners' evaluation

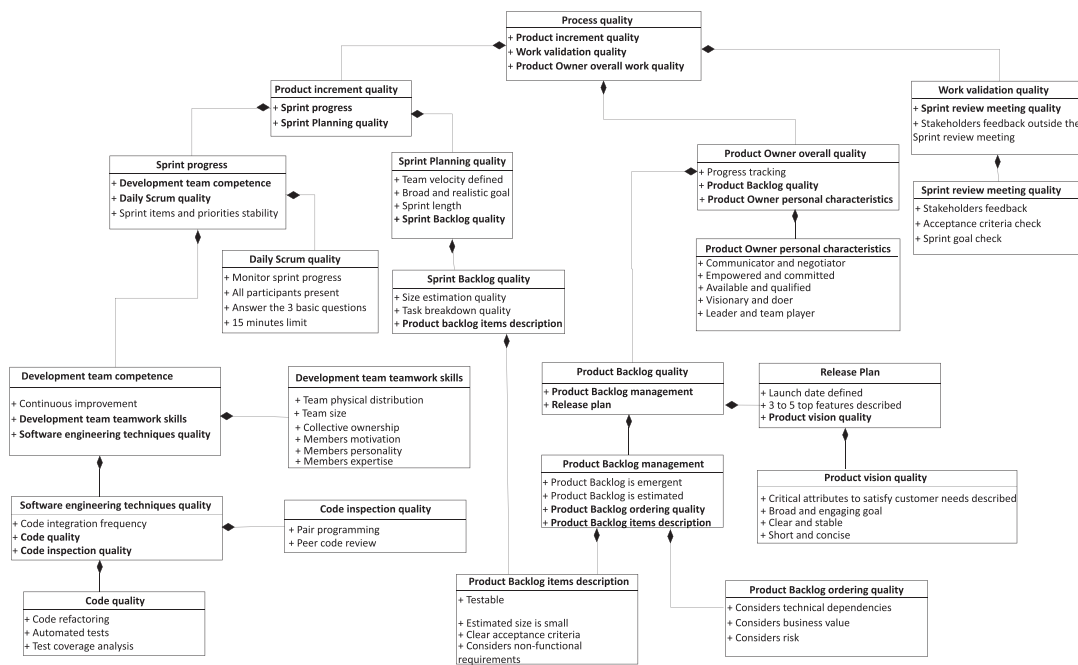


FIGURE 21 First experience's complete class diagram

improvement opportunities. The method³⁸ that we used to define the NPT was criticized given its subjectivity and its modeling limitations. It is based on ordinal (i.e., ranked) nodes as presented in Fenton et al,²³ which presents 4 types of functions to model the NPTs. We only used one, which limits modeling capabilities.

Regarding the variables' scale, we had previously agreed with using a 3-point Likert scale, as shown in Section 4.2. On the other hand, by applying the model to real projects, we observed that only 3 states limit the modeling capabilities due to lack of options to represent the reality of the projects, which is evidence of a failure in our previous process to define the number of states for each node.

Furthermore, for some NPTs, node divorcing,²⁴ which consists of separating the parent nodes from one another, can be applied to reduce the complexity of defining the NPTs and the effects of combinatorial explosion. For instance, the factor *Development team teamwork skills* depends directly on 6 factors. Considering that *Development team teamwork skills* is defined with a 3-point scale, its NPT will have 3⁶ (i.e., 729) cells.

According to Williams,³⁶ our first model is missing 2 important agile practices: *Definition of Done* (i.e., done criteria) and *Emergent Design*. *Definition of Done* is presented in the Scrum Guide³ and considered an important practice to maintain focus on the product quality.³⁶ Regarding *Emergent Design*, even though we did not consider it explicitly, we consider *Code refactoring* as an indicator for the given factor.

Finally, our first model is mainly composed of subjective metrics, except for test coverage. For process and product quality analysis of agile projects to be automated, many objective metrics have been proposed such as Running Tested Feature and percentage of automated tests.^{41,48} Replacing some subjective metrics with objective metrics would decrease the effort to input data to the model, because the values of some factors could be defined automatically. Furthermore, it would reduce the model's subjectivity, because some of the factor's values would be independent of the *Scrum Master's* judgment.

5 | PROCESS TO BUILD THE SECOND VERSION OF THE BAYESIAN NETWORK

With the purpose of solving the problems described in Section 4.3, we improved the model presented in Section 4. We started by evaluating each fragment in the Bayesian network to identify improvement opportunities. Afterwards, the first step, which we present in Section 5.1, was to add essential agile practices as presented in Williams³⁶ and refactor the DAG to reduce the complexity of defining the NPTs. The second step, which we present in Section 5.2, was to define the NPT following the method presented in Fenton et al,²³ which, compared with the method of Perkusich et al³⁸ used to build our first model, improves the modeling capabilities (i.e., uses 4 types of weighted expressions to define NPT). Finally, as presented in Section 5.3, we validated the model with simulated scenarios.

5.1 | DAG revision

The first step was to add the factor *Definition of Done*, considered by Williams,³⁶ one of the most important agile practice. According to Sutherland and Schwaber,³ *Definition of Done* is an artifact to ensure a shared understanding of when a *Product Backlog* item or an increment is done. So, in our model, it would be an attribute of the class *Product increment quality*. Instead of defining the class *Definition of Done* as an instance variable of *Product increment quality*, we created a new class *Scope definition quality* and defined *Definition of Done* and *Sprint Planning quality* as its instance variables. Afterwards, we defined a composition between *Product increment quality* and *Scope definition quality*. The reasoning behind this decision was that, for a given *sprint*, the total amount

of work that should be performed by the team is the combination of tasks related to the functional goals of the *sprint* and the quality criteria defined in the *Definition of Done*. Furthermore, this configuration reduces the complexity of defining the NPTs. We show the resulting class diagram in Figure 22.

Afterwards, we noticed that the fragment *Software engineering techniques quality* could be improved. The first modification was to modify *Code inspection quality* and define it as an attribute of *Code quality* instead of *Software engineering techniques quality quality*, because code inspection activities are an indicator of the source code quality. Afterwards, we removed *Code quality's* attribute *Documentation*, because it is an attribute of the source code quality that is controlled by code inspection activities. So, if we kept it, we would be representing the same factor (i.e., documentation) twice.

In general, there are 2 types of automated tests: acceptance and unit tests. Given this, we added 2 metrics to indicate *Automated tests*: *Running Tested Features* and *Unit tests coverage*. Depending on the project's context, other metrics could be used. Furthermore, we removed *Pair programming* from the model, because we concluded that it is an "extreme" case of *Peer code review*. We show the resulting class diagram in Figure 23.

The second step was to modify the DAG to reduce the model complexity and improve performance. For this purpose, we used *divorcing*,²⁴ which consists of applying the synthesis idiom to cut down the combinatorial space. We show the modifications performed using divorcing in Figures 24 to 26.

Afterwards, we removed *Development team competence's* attribute *Continuous improvement*, because the model's goal is to assist on

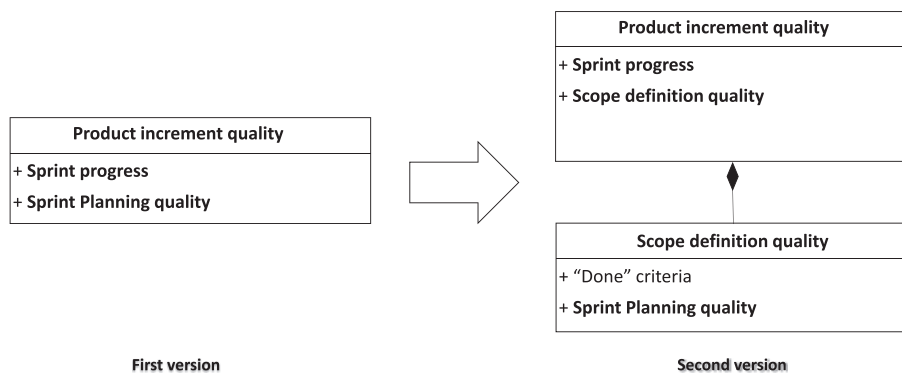


FIGURE 22 *Product increment quality* *Product increment quality* class diagram modifications

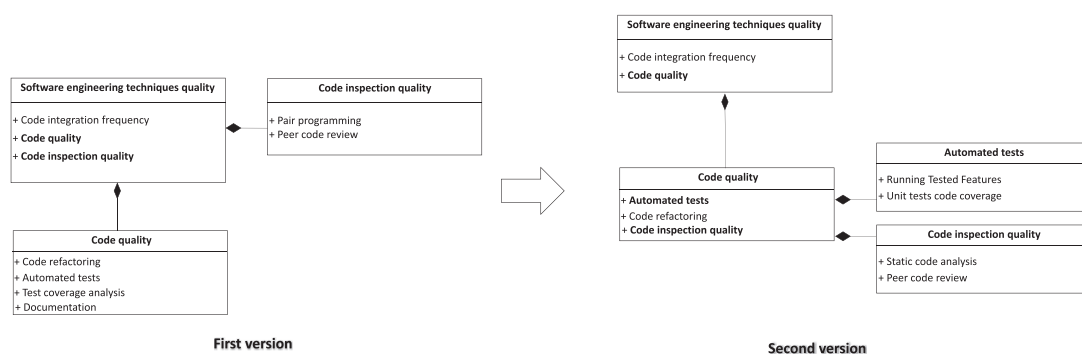


FIGURE 23 *Software engineering techniques quality* *Software engineering techniques quality* class diagram modifications

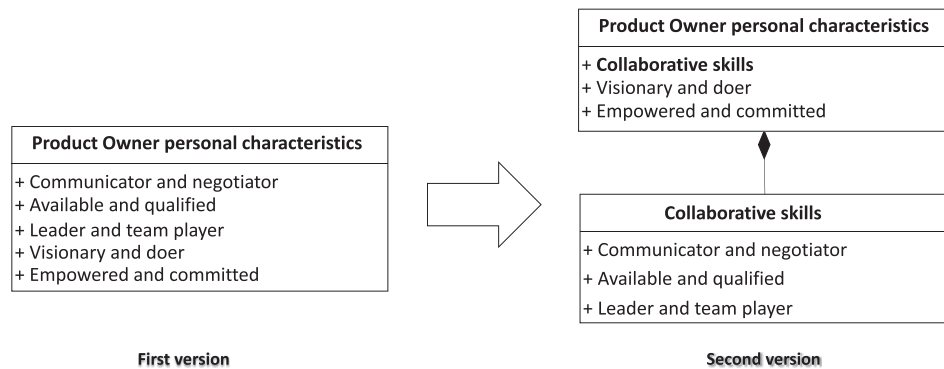


FIGURE 24 Modifications in *Product Owner personal characteristics* caused by applying the divorcing technique

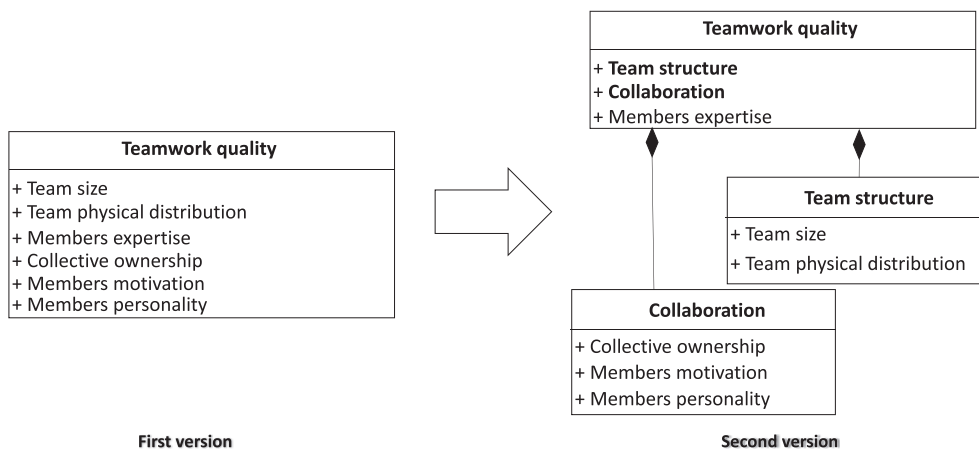


FIGURE 25 Modifications in *Teamwork quality* caused by applying the divorcing technique

continuous improvement. So we noticed after evaluating the model in real projects¹⁰ that it was confusing to input values for this factor. Finally, to improve modeling capabilities, we changed from using 3-point Likert scale to 5-point Likert scale.

5.2 | NPT definition

For the NPT of a ranked node to be defined, it is necessary to define 3 parameters: f , $V = (v_1, \dots, v_k)$ and σ^2 , where f is the type of function, V is a vector containing parent node's weights, k is the number of parent nodes, and σ^2 is the variance. In AgenaRisk, the tool used to execute the Bayesian network, these variables have the following range: $f \in \{WMEAN, WMIN, WMAX, MIXMINMAX\}$, $w \in \{1, \dots, 5\}$, $\sigma^2 \in \{5.0E-4, \dots, \infty\}$ and $k \in \{1, \dots, \infty\}$.

With this purpose, we applied the method presented in Fenton et al.,²³ which consists of eliciting data from an expert through “what if” analysis and truth table. We elicited data from an expert with 5 years of experience as *Scrum Master* and was already familiarized with the model presented in Perkusich et al.¹⁰ Furthermore, for each dependent node in the DAG, we defined a set of questions to elicit knowledge from the expert. Each question was defined in terms of combinations of states of the given node's parents. The set of questions defined the number of lines of the truth table and the data elicited from the expert, the result for each line. Given that the knowledge from the expert was elicited, we defined the f , V , and σ^2 through reasoning and trial and error until we

reached accurate inferences. Initially, we defined, for each node, $2 \times i$ questions, where i is the number of parents. The questions were defined using extreme values, such as the combination *Very low* and *Very high*, using a 5-point Likert scale. If, for a given case, the number of questions was insufficient to define the f , V , and σ^2 , we added questions.

To assess the accuracy of the calculations (i.e., inferences), we used the Brier score.⁴⁹ For a single prediction, which is our case, the Brier score is simply the square of the difference between the predicted probability (q) and the actual outcome (o),²⁴ for each state: $B = \sum_{n=1}^s (o_n - q_n)^2$, where B is the Brier score and s is the number of possible outcomes (i.e., number of states of the given node). Given that we want the best possible calibration, we attempted to, given the data collected from the experts, find a combination of parameters f and $V = (v_1, \dots, v_n)$ that minimizes B . We considered an inference to be accurate with a mean Brier score less or equal to 0.1. For brevity, in this paper, we only show the process to define 1 NPT. We created a webpage[†] to present the data collected to define all the NPTs.

Product increment quality has 2 attributes: *Sprint progress* and *Scope definition*. Therefore, we created a truth table with the possible combination of the extreme values of both attributes to perform “what if” analysis with the expert. We present the truth table in Table 2. By analyzing Table 2, we concluded that if any of the attribute's value is low, *Product increment quality* tends to be low. Therefore, we decided to use

[†]<https://mirkoperkusch.wordpress.com/2015/05/18/second-experience-npt-calibration-data/>

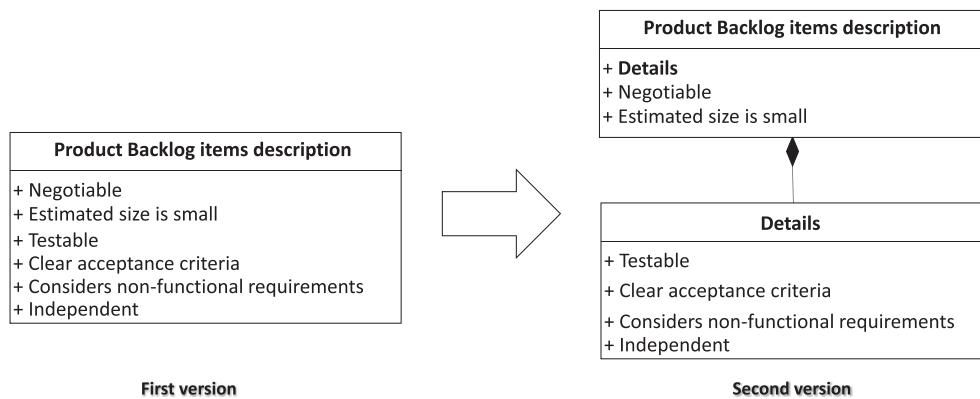


FIGURE 26 Modifications in *Product Backlog items description* caused by applying the divorcing technique

TABLE 2 Truth table for *Product increment quality*

A	B	C
Very high	Very high	Very high
Very low	Very low	Very low
Very low	Very high	Low
Very high	Very low	Low

WMIN function with the same weights for both attributes. Empirically, we chose the weights to be 5 and $\sigma^2 = 0.0005$, because we observed that WMIN tends to be a minimum function reflecting the expert's confidence in the results with this configuration.

5.3 | Validation

Because we had success in validating the first version of the *Bayesian network*, to test the new version we used the same 10 simulated scenarios that we used to validate the first one and compared the results. A simulated scenario is defined in terms of a scenario description, input values for the independent nodes and expected values for the highest level nodes.

The simulated scenarios were defined by a *Scrum Master*, which acted as an agile coach in the company that we performed the case studies for the first *Bayesian network*. For each scenario, he defined the expected results given his experience. The expected results were, initially, defined using only words. Afterwards, for the highest level nodes, the expected calculated values (i.e., hypotheses) were defined.

For brevity, this paper limits itself to present the analysis of only one of the scenarios used to test the model. We created a webpage⁵ to present all the calculated data from the scenarios. This scenario represents a *Scrum* software development project that has a highly capable and organized development team, but does not have a person committed and skilled to play the role of the *Product Owner*. Furthermore, the stakeholders do not collaborate closely with the team. The expected result for this project is to have a small chance of reaching its goals, and the major reasons for the failure are the *Product Owner* low work quality and lack of proper work validation. In Table 3, we present the expected values for the highest level nodes.

TABLE 3 Expected values for highest level nodes in the first version of the *Bayesian network*

Node	States, %		
	Bad	Moderate	Good
Project progress	100	0	0
Product Owner overall work quality	50	50	0
Product increment quality	0	100	0
Work validation quality	100	0	0

TABLE 4 Scenario results for highest level nodes in the first version of the *Bayesian network*

Node	States, %		
	Bad	Moderate	Good
Project progress	6	86	8
Product Owner overall work quality	20	77	3
Product increment quality	1	62	37
Work validation quality	13	85	2

TABLE 5 Scenario results for highest level nodes in our second version of the *Bayesian network*

Node	States, %		
	Bad	Moderate	Good
Project progress	80	20	0
Product Owner overall work quality	54	46	0
Product increment quality	15	83	2
Work validation quality	80	20	0

After defining this scenario, we defined the values for the input nodes in the model and ran it on AgenaRisk. In Table 4, we present the values for the nodes *Project progress* and its parent nodes, which are the most important nodes for the initial analysis, for the first version of the *Bayesian network*. In Table 5, we present the values for the second version. Even though we used 5-point scale in the second version, in both tables, to ease the comparison, we present data using 3-point scale, as in the first version.

To compare the accuracy of the both models, we compared the mean Brier score given the 4 nodes highest level nodes. For the first version, the mean Brier score was 0.558; for the second version, 0.002. Since the Brier score is lower for the second version, we can conclude that it is more accurate in this scenario.

⁵<https://mirkoperkusich.wordpress.com/2015/05/18/second-experience-validation-results/>

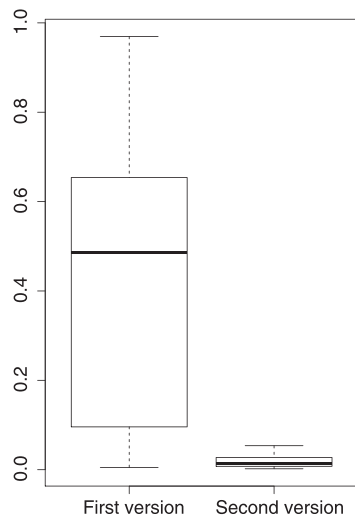


FIGURE 27 Boxplot of the mean Brier score for all scenarios

We followed the same process for the remaining scenarios. In Figure 27, we present a boxplot of the mean Brier score for all scenarios. By analyzing the boxplot, we can conclude that the new version has a better accuracy.

By analyzing both tables, we concluded that the results from the second model are better, because it emphasizes with more confidence that the project progress is not going well, as it was expected, and the biggest reasons for it are the bad work validation process and bad quality of work performed by the Product Owner. By analyzing the values of the nodes *Work validation*, *Product Owner work quality*, and their parents, the *Scrum Master* can observe which problems are the major sources for having the current values on the node *Project progress* and use this information to lead the project to improve the chances of succeeding.

5.4 | Threats to validity

The internal and construct threats to validity are related to the NPTs calibration. Because the *Bayesian network* is large, the expert might have reacted differently as time passed. Furthermore, the expert was chosen because of availability, not randomly. Therefore, the fact that we were able to, given the collected data, calibrate the NPTs, might have been coincidence. Furthermore, because the NPTs were calibrated by only 1 person, they are biased by his experience and opinions. The external threats to validity are related with the simulated scenarios defined to validate the model. Because they were defined by 1 person, they might not be enough to generalize that the model is suited to detect process problems in all situations of *Scrum* projects. On the other hand, because we performed a case study with the first version of the *Bayesian network*, it is an indicator that our models can help *Scrum* teams to continuously improve.

6 | CONCLUSION

In this paper, we presented the process to build a *Bayesian network* to assist on empirical software process management of *Scrum* projects. The goal of the proposed model is to assist on the continuous

improvement by considering technical and managerial best practices and process metrics. *Scrum Masters* should use them to perform assessment and predictions to gain knowledge and help the team to improve.

We presented details regarding the process executed to build the first version of the *Bayesian network*, which was evaluated with an industrial case studies, as presented in Perkusich et al.¹⁰ We discussed limitations of the first model such as not considering key agile practices, metrics, and unnecessary complexity to define the *Bayesian network's* NPT. Furthermore, we presented a new model that we built by modifying the first version and implementing improvements. We successfully validated it with 10 simulated scenarios and concluded that it adequately models *Scrum* projects from the viewpoint of the *Scrum Master*.

In a survey presented by VersionOne,² the leading cause (44%) of failed agile projects is the lack of experience with agile methods. Therefore, we encourage practitioners to use the model as a tool to assess the process and identify early the factors that need improvement. Furthermore, in the same study, 42% of the respondents point to company culture at odds with core agile values as the leading cause of failed projects. By using the models presented in this paper, practitioners can collect and record data of agile projects as a source of information to show agile advantages to upper management and change the company's culture.

In this paper, we presented generic models, because they consider best practices of the industry. However, for a practice to be effective, it depends on the context of the project (e.g., organization culture). To adapt to a specific context, practitioners recommend agile processes tailoring.⁴⁸ As presented in Section 5, the second version of the *Bayesian network* was built by modifying the first one and its inferences have improved accuracy. This shows that using *Bayesian networks* as the modeling tool allows the model to be easily modified. This way, we conclude that the model is flexible and in conformance with process tailoring, which is a common practice in the software industry.

Even though we applied more metrics that can be automatically collected by tools (i.e., objective) in our second experience, their interpretation is still subjective. In this case, by interpretation, we mean translating the metric from a numeric scale to an ordinal scale. For instance, define that 35% of test coverage is *Bad*. This is due to the fact that the thresholds that were defined in the numeric scale and used to convert it to an ordinal scale were set manually given the practitioner's experience (i.e., subjective). Given this, we need to evaluate using numeric variables in the *Bayesian network* or applying techniques such as the ones presented in Foucault et al.⁵⁰ and Shatnawi⁵¹ to automatically define thresholds given historical data and statistics.

Currently, we are working on automating the process of building NPTs to reduce the subjectivity and performing case studies in real projects to validate our second model. For future works, we intend to integrate the model construction with an agile model tailoring framework such as AM-QuICK.⁴⁸ Furthermore, we intend to integrate our model with performance measurement models such as the ones presented in Hearty et al.,⁶ which forecasts productivity, and Wagner et al.,⁵² which assesses the product quality. With this, we will unify continuous improvement and key performance indicators assessment.

REFERENCES

- Jinzenji K, Hoshino T, Williams L, Takahashi K. An experience report for software quality evaluation in highly iterative development methodology using traditional metrics. *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*; November 2013: 310–319.
- VersionOne. 9th annual state of agile development survey results. <http://www.versionone.com/pdf/state-of-agile-development-survey-ninth.pdf> Accessed May 5, 2015; 2015.
- Sutherland J, Schwaber K. The Scrum Guide. <http://www.scrumguides.org/docs/scrumguide/v1/Scrum-Guide-US.pdf> Accessed March 2, 2016; 2013.
- Schwaber K. Scrum is hard and disruptive. <http://www.verheulconsultants.nl/ScrumsHardandDisruptive.pdf> Accessed November 26, 2013; 2006.
- Ziv H, Richardson DJ. Constructing Bayesian-network models of software testing and maintenance uncertainties. *International Conference on Software Maintenance*, 1997. *Proceedings*, Bari, Italy; October 1997:100–109.
- Hearty P, Fenton N, Marquez D, Neil M. Predicting project velocity in XP using a learning dynamic Bayesian network model. *IEEE Trans Softw Eng*. 2009;35(1):124–137.
- de Melo ACV, Sanchez AJ. Software maintenance project delays prediction using Bayesian networks. *Expert Syst Appl*. 2008;34(2):908–919.
- Hu Y, Mo X, Zhang X, Zeng Y, Du J, Xie K. Intelligent analysis model for outsourced software project risk using constraint-based bayesian network. *J Syst Software*. 2012;7(2):440–449.
- Perkusich M, de Almeida HO, Perkusich A. *Proceedings of the 28th Annual ACM Symposium on Applied Computing, SAC '13*, New York, NY, USA; 2013:1037–1042. ACM.
- Perkusich M, Soares G, Almeida H, Perkusich A. A procedure to detect problems of processes in software development projects using Bayesian networks. *Expert Syst Appl*. 2015;42(1):437–450.
- Cohn M. *Succeeding With Agile: Software Development Using Scrum*. 1st ed. Boston, MA, USA: Addison-Wesley Professional; 2009.
- Anderson DJ. *Successful Evolutionary Change for Your Technology Business*. 1st ed. Seattle, WA, USA: Blue Hole Press; 2010.
- Griffiths M. *PMI-ACP Exam Prep*. Minnetonka, MN, USA: RMC (Rita Mulcahy Companies) Publications; 2012.
- Pichler R. *Agile Product Management With Scrum: Creating Products That Customers Love*. 1st ed. Boston, MA, USA: Addison-Wesley Professional; 2010.
- Ben-Gal I. Bayesian networks. In: Ruggeri F, Kenett R, Faltin FW, eds. *Encyclopedia of Statistics in Quality and Reliability*. New Jersey, NY, USA: John Wiley and Sons; 2007.
- Friedman N, Geiger D, Goldszmidt M. Bayesian network classifiers. *Mach Learn*. 1997;29(2-3):131–163.
- Pearl J, Russell S. Bayesian networks. *Handbook of Brain Theory and Neural Networks*, Cambridge, MA, USA: MIT Press; 1995.
- Uusitalo L. Advantages and challenges of Bayesian networks in environmental modelling. *Ecol Model*. 2007;203(34):312–318.
- Lee E, Park Y, Shin JG. Large engineering project risk management using a Bayesian belief network. *Expert Syst Appl*. April 2009;36(3):5880–5887.
- Zhang L, Wu X, Ding L, Skibniewski MJ, Yan Y. Decision support analysis for safety control in complex project environments based on Bayesian networks. *Expert Syst Appl*. 2013;40(11):4273–4282.
- de Oliveira MA, Possamai O, Dalla Valentina LV, Flesch CA. Applying Bayesian networks to performance forecast of innovation projects: a case study of transformational leadership influence in organizations oriented by projects. *Expert Syst Appl*. 2012;39(5):5061–5070.
- Neil M, Fenton N, Nielson L. Building large-scale Bayesian networks. *Knowl Eng Rev*. September 2000;15(3):257–284.
- Fenton NE, Neil M, Caballero JG. Using ranked nodes to model qualitative judgments in Bayesian networks. *IEEE Trans Knowl Data Eng*. October 2007;19(10):1420–1432.
- Fenton N, Neil M. *Risk Assessment and Decision Analysis With Bayesian Networks*. 5th ed. Boca Raton, FL, USA: CRC Press; 2012.
- Fan C-F, Yu Y-C. BBN-based software project risk management. *J Syst Software*. October 2004;73(2):193–203.
- Fenton N, Krause P, Neil M. Software measurement: uncertainty and causal modeling. *IEEE Software*. 2002;19(4):116–122.
- Jeet K, Bhatia N, Minhas RS. A model for estimating the impact of low productivity on the schedule of a software development project. *SIGSOFT Softw Eng Notes*. August 2011;36(4):1–6.
- Aboueila M, Benedicenti L. Bayesian network based XP process modelling. *Int J Softw Eng Appl*. 2010;1(3):1–15.
- Jeet K, Bhatia N, Minhas RS. A Bayesian network based approach for software defects prediction. *SIGSOFT Softw Eng Notes*. 2011;36(4):1–5.
- Pendharkar PC, Subramanian GH, Rodger JA. A probabilistic model for predicting software development effort. *IEEE Trans Softw Eng*. 2005;31(7):615–624.
- Settas D, Bibi S, Sfetsos P, Stamelos I, Gerogiannis V. Using bayesian belief networks to model software project management antipatterns. *Fourth International Conference on Software Engineering Research, Management and Applications*, Seattle, Washington, USA, 2006; 2006:117–124.
- Stamelos I. Software project management anti-patterns. *J Syst Software*. 2010;83(1):52–59.
- Stamelos I, Angelis L, Dimou P, Sakellaris E. On the use of Bayesian belief networks for the prediction of software productivity. *Inf Softw Technol*. 2003;45(1):51–60.
- Nagy A, Njima M, Mkrtchyan L. A Bayesian based method for agile software development release planning and project health monitoring. *2010 2nd International Conference on Intelligent Networking and Collaborative Systems (INCOS)*, Thessaloniki, Greece; 2010:192–199.
- Eloranta V-P, Koskimies K, Mikkonen T. Exploring Scrumbutan empirical study of Scrum anti-patterns. *Inf Softw Technol*. 2016;74:194–203.
- Williams L. What agile teams think of agile principles. *Commun ACM*. 2012;55(4):71–76.
- Campanelli AS, Parreiras FS. Agile methods tailoring a systematic literature review. *J Syst Software*. 2015;110:85–100.
- Perkusich M, Perkusich A, Almeida HO. Using survey and weighted functions to generate node probability tables for Bayesian networks. *2013 BRICS Congress on Computational Intelligence and 11th Brazilian Congress on Computational Intelligence*, Ipojuca, Brazil; 2013:183–188.
- Project Management Institute. *PMBok: Project Management Body of Knowledge*. 5th ed. Newtown Square, PA, USA: Project Management Institute; 2013.
- Grapenthin S, Poggel S, Book M, Gruhn V. Improving task breakdown comprehensiveness in agile projects with an interaction room. *Inf Softw Technol*. 2015;67:254–264.
- Kupiainen E, Mntyl MV, Itkonen J. Using metrics in agile and lean software development a systematic literature review of industrial studies. *Inf Softw Technol*. 2015;62:143–163.
- Stray V, Sjöberg DIK, Dyb T. The daily stand-up meeting: a grounded theory study. *J Syst Software*. 2016;114:101–124.
- Stray VG, Moe NB, Dybå T. In: Wohlin C, ed. *Agile Processes in Software Engineering and Extreme Programming: 13th International Conference, XP 2012, Malmö, Sweden, May 21–25, 2012. proceedings*, chapter Escalation of Commitment: A Longitudinal Case Study of Daily Meetings. Berlin, Heidelberg: Springer Berlin Heidelberg; 2012:153–167.
- Fontana RM, Fontana IM, da Rosa Garbuio PA, Reinehr S, Malucelli A. Processes versus people: how should agile software development maturity be defined? *J Syst Software*. 2014;97:140–155.
- Beck K. *Extreme Programming Explained: Embrace Change*. 1st ed. Boston, MA, USA: Addison-Wesley; 2000.

46. Petersen K, Wohlin C. A comparison of issues and advantages in agile and incremental development between state of the art and an industrial case. *J Syst Software*. 2009;82(9):1479–1490. SI: {QSIC} 2007.
47. Smart Software Inc. Peer code review: an agile process. https://support.smartbear.com/support/media/resources/cc/Peer-Code-Review_An-Agile-Process.pdf. Accessed April 18, 2013; 2013.
48. Ayed H, Habra N, Vanderose B. Am-quick: a measurement-based framework for agile methods customisation. *2013 Joint Conference of the 23rd International Workshop on Software Measurement and the 2013 Eighth International Conference on Software Process and Product Measurement (IWSM-MENSURA)*, Ankara, Turkey; 2013:71–80.
49. Brier GW. Verification of forecasts expressed in terms of probability. *Mon Weather Rev*. 1950;78(1):1–3.
50. Foucault M, Palyart M, Falleri J-R, Blanc X. Computing contextual metric thresholds. *Proceedings of the 29th Annual ACM Symposium on Applied Computing*, SAC '14. ACM, New York, NY, USA; 2014:1120–1125.
51. Shatnawi R. Deriving metrics threshold using log transformation. *J Softw Evol Proc*. 2015;27:95–113.
52. Wagner S, Lochmann K, Heinemann L, Klas M, Trendowicz A, Plosch R, Seidi A, Goeb A, Streit J. The Quamoco product quality modelling and assessment approach. *2012 34th International Conference on Software Engineering (ICSE)*, Zurich, Switzerland; 2012:1133–1142.

How to cite this article: Perkusich M, Gorgônio KC, Almeida H, Perkusich A. Assisting the continuous improvement of scrum projects using metrics and Bayesian networks. *J Softw Evol Proc*. 2017;29:e1835. <https://doi.org/10.1002/smr.1835>